Shivansh Agarwal

cs25mtech14013@iith.ac.in | shivansh919@gmail.com | +91 7668128307 | LinkedIn | GitHub

EDUCATION

Institution	Degree / Board	Passing Year	Score / CGPA
Indian Institute of Technology Hyderabad	B.Tech	2027	8.40
AVMD Institute	Class XII (Central Board of Secondary Education)	2018	80.20%
St. Georges College	Class X (Indian Certificate of Secondary Education)	2016	87.40%

EXPERIENCE

Intel Labs | Networking System Engineer

- Contributed to **Pronto, a $30M DARPA-funded programme**, alongside researchers from **ONF and the Linux Foundation** to improve the efficiency and performance of next-generation 5G cellular networks.
- Built an **automated test pipeline** across ONOS and RIC repositories and integrated Dependabot, raising close to **200 pull requests** and keeping dependencies patched without manual tracking.
- Extended the xApp suite for RAN control, work later taken up by China Unicorn and validated through a proof-of-concept deployment over live radios serving real subscribers.
- **Tech Stack: C, Docker, Kubernetes, GitHub Actions, Shell Scripting.**

Intel | Undergraduate Intern

- **Cut xApp latency by 25-80%** through a cross-functional optimization effort with the PRC team, improving the responsiveness of live radio resource allocation under peak traffic.
- Delivered the **Network Slicing Radio Resource Management xApp**, which shipped on **Intel's Edge Software Hub** and became available to external telecom customers as a product.
- Developed the **xApp user interface** in **React.js and Node.js** to visualize slicing behaviour in real time, enabling leadership to review the product without reading raw logs.
- Championed engineering best practices such as **unit-test coverage and documented ERDs**, which improved reliability and made the codebase easier for new contributors to extend.
- **Tech Stack: C, React.js, Node.js, Docker, Kubernetes, Helm Charts, Shell Scripting.**

THESIS (ONGOING)

Rethinking Prefetching for STT-RAM Last-Level Caches

- Extended the **ChampSim** microarchitectural simulator with a **configurable STT-RAM last-level cache** modelling read/write latency asymmetry (**8× costlier writes**; iso-area **4 MB SRAM -> 16 MB STT-RAM**) to benchmark **6 hardware prefetchers** (SMS, AMPM, Best-Offset, SPP, Bingo, IPCP) across **single-core and 4-core** workloads.
- Demonstrated that SRAM-tuned prefetchers **lose nearly all of their benefit** on an STT-RAM LLC: single-core geomean speedup falls from **1.45× to 1.05×**, while 4-core prefetching drops **below the no-prefetch baseline**

(**0.86-0.92× weighted speedup**), actively degrading performance.

- Root-caused the regression to **write amplification**: despite a **higher LLC hit rate**, write-heavy over-prediction (useless prefetches up to ~**30× baseline misses**) saturates the slow STT-RAM write path, isolated through a coverage / accuracy / over-prediction breakdown.
- Ran the full sweep over **100 workload traces** spanning **SPEC CPU 2017/2006, PARSEC, Ligra and CVP**, classifying every workload as **prefetcher-friendly, prefetcher-adverse or prefetcher-neutral** to analyse prefetcher behaviour per category.
- Designing an **NVM-write-aware prefetching** policy that co-tunes prefetch aggressiveness with the STT-RAM LLC to recover the lost speedup without paying the write penalty.
- **Tech Stack: C++, ChampSim, Hardware Prefetching, NVM/STT-RAM**

PROJECTS

Write-Optimized Crash-Consistent B+Tree for NVM | C++17, x86 Persistence Intrinsics, mmap, Linux

- Applied the NVM write-cost insight from my thesis one layer up in software: a persistent B+Tree that keeps leaf entries **unsorted behind an 8-byte validity bitmap**, cutting persisted cache lines per insert from **5.5-16.6 (rising with leaf occupancy) to a constant 2** with **no undo/redo logging**, since the bitmap flip is a single atomic durable store.
- Combined **selective persistence** (volatile inner nodes, persistent leaves) with **1-byte key fingerprints** to hold lookup throughput at **85-112% of a sorted-leaf baseline** despite the unsorted layout, rebuilding the inner tree on restart in **44 ms for 1.26M keys**.
- Emulated STT-RAM / Optane-class read-write asymmetry with an **mmap-backed persistence shim** instrumenting every clwb/sfence and injecting **up to 600 ns of write latency per drained queue batch**, measuring **21-49% higher insert throughput** than a naive persistent B+Tree at comparable read performance.
- Proved durability with a **fault-injection harness** that crashes and recovers the index at **each of 32,957 persist points** under randomized workloads, verifying every acknowledged key survives and no torn node state is observable after recovery.

ACHIEVEMENTS

- **Received Intel's Recognition Award for strengthening the company's partnership with its customers through sustained delivery on customer-facing cellular network software.**
- **Achieved a maximum rating of 4 stars on CodeChef and the Knight badge on LeetCode by competing regularly in timed algorithmic contests.**

SKILLS

Languages: C, C++.

Technologies and Frameworks: Git, Docker, Kubernetes, Helm Charts, GitHub Actions, React.js, Node.js